

Metodi AI per la Fisica - Applicazioni Fisiche al ML 2026

Progetto di gruppo

Colorazione di grafi con Graph Neural Networks e minimizzazione differenziabile dell'energia di Potts

1 Obiettivo del progetto

Lo scopo del progetto è implementare un metodo basato su **Graph Neural Networks** per affrontare il problema della **colorazione di grafi**.

Dato un grafo non orientato $G = (V, E)$ e un numero intero c , il problema consiste nel determinare se sia possibile assegnare a ogni nodo $i \in V$ uno tra c colori disponibili in modo tale che due nodi adiacenti non abbiano mai lo stesso colore. In altre parole, per ogni arco $(i, j) \in E$, deve valere:

$$\text{color}(i) \neq \text{color}(j).$$

Il progetto richiede di formulare questo problema come un problema di ottimizzazione differenziabile, utilizzando una GNN per produrre una rappresentazione continua dell'assegnazione dei colori ai nodi e minimizzando una funzione di loss ispirata all'hamiltoniana antiferromagnetica del modello di Potts. In altre parole il progetto non richiede di allenare una GNN che poi possa essere applicata ad un grafo generico in input, ma di allenare una GNN per ogni grafo di input, eg determinare i parametri della GNN che minimizzano l'energia di Potts dello specifico grafo in input.

2 Descrizione del metodo richiesto

Ogni gruppo dovrà implementare un algoritmo che riceve in input un grafo e un numero di colori c , e restituisce una stima del fatto che il grafo sia o meno colorabile con c colori.

Il grafo dovrà essere codificato come struttura dati adatta a una GNN, per esempio tramite matrice di adiacenza, lista degli archi, tensori PyTorch o librerie dedicate alla manipolazione di grafi.

A ogni nodo del grafo dovrà essere associato un embedding, aggiornato tramite una semplice architettura di **message passing**. La GNN dovrà produrre, per ogni nodo, un vettore di dimensione c , interpretabile come assegnazione "soft" del nodo ai c colori disponibili. Una possibile scelta è applicare una funzione softmax all'output finale della rete, ottenendo per ogni nodo i un vettore:

$$p_i = (p_{i1}, p_{i2}, \dots, p_{ic}),$$

dove p_{ik} rappresenta il peso o la probabilità associata al colore k .

La funzione di loss dovrà essere costruita come versione differenziabile dell'energia del modello di Potts. L'idea è penalizzare gli archi che connettono nodi ai quali la rete tende ad assegnare lo stesso colore. Una possibile forma della loss è:

$$\mathcal{L} = \sum_{(i,j) \in E} p_i \cdot p_j = \sum_{(i,j) \in E} \sum_{k=1}^c p_{ik} p_{jk}.$$

Questa quantità è piccola quando nodi adiacenti hanno assegnazioni di colore diverse, ed è grande quando nodi collegati tendono ad avere lo stesso colore.

L'ottimizzazione dovrà essere effettuata usando la differenziazione automatica e gli ottimizzatori disponibili in PyTorch, per esempio Adam o SGD.

Al termine dell'ottimizzazione, gli studenti dovranno ottenere una colorazione discreta assegnando a ogni nodo il colore corrispondente alla componente massima del vettore di output:

$$\text{color}(i) = \arg \max_k p_{ik}.$$

La qualità della soluzione dovrà essere valutata contando il numero di conflitti residui, cioè il numero di archi (i, j) tali che:

$$\text{color}(i) = \text{color}(j).$$

Una colorazione è considerata valida se il numero di conflitti è zero.

3 Dataset

Ogni gruppo ha accesso ha a disposizione 2 grafi, in formato DIMACS standard, scaricabili dal sito github del corso (link: https://github.com/stefanogiagu/corso_AI_2026/tree/main/grafi).

Il nome dei due grafi da utilizzare per ogni gruppo è indicato nel file excel dei gruppi (link: https://docs.google.com/spreadsheets/d/1C5sijeqpI40shyjyUTYF51EqShqIo22_2K4Kd8fT4CA/edit?usp=sharing).

Per leggere e convertire in un grafo pytorch geometric il grafo in formato DIMACS potete usare l'esempio di codice python disponibile al link: https://github.com/stefanogiagu/corso_AI_2026/blob/main/ReadGraph_example.ipynb.

4 Attività richieste

Ogni gruppo dovrà:

1. Implementare una pipeline completa per leggere o costruire i grafi forniti.
2. Implementare una semplice Graph Neural Network basata su message passing o graph convolution.
3. Associare ai nodi una rappresentazione continua dei colori tramite embedding o vettori di probabilità.
4. Definire una funzione di loss differenziabile ispirata all'hamiltoniana di Potts.
5. Minimizzare la loss rispetto ai parametri della rete usando PyTorch.
6. Convertire l'output continuo della rete in una colorazione discreta.
7. Valutare il numero di conflitti residui.
8. Ripetere l'esperimento per diversi valori del numero di colori c , in modo da stimare il numero cromatico dei grafi assegnati.
9. Applicare il metodo ai due grafi forniti.
10. Documentare il lavoro in un notebook Jupyter/Colab contenente codice eseguibile, descrizione metodologica, risultati numerici e grafici.

5 Stima del numero cromatico

Per ciascuno dei due grafi forniti, gli studenti dovranno provare diversi valori di c .

Per ogni valore di c , dovranno riportare:

$$N_{\text{conflicts}}(c)$$

cioè il numero di archi in conflitto dopo la discretizzazione della soluzione.

Una possibile procedura è:

- partire da un valore piccolo di c ;
- addestrare la GNN;
- valutare il numero di conflitti;
- aumentare c finché si ottiene una colorazione valida;
- ripetere eventualmente più volte l'ottimizzazione per ogni c , usando inizializzazioni casuali diverse.

Il numero cromatico stimato sarà il più piccolo valore di c per cui il metodo riesce a trovare una colorazione senza conflitti.

Gli studenti dovranno discutere chiaramente se il risultato ottenuto è una dimostrazione certa o una stima algoritmica.

6 Relazione richiesta nel notebook

Il notebook finale dovrà contenere sia il codice sia una relazione scientifica breve ma completa. In particolare, dovranno essere presenti le seguenti sezioni: Introduzione al problema, Strategia Utilizzata, Architettura della rete, Dettagli del training, Risultati, Discussione.

Nella discussione critica dei risultati ottenuti si dovrebbe come minimo rispondere alle seguenti domande:

- il metodo converge sempre?
- quanto dipende dall'inizializzazione casuale?
- quali valori di c risultano sufficienti?
- ci sono casi in cui la loss diminuisce ma rimangono conflitti?
- quali sono i limiti del metodo?
- quali miglioramenti sarebbero possibili?

7 Consegna

La consegna consiste in un notebook Jupyter/Colab eseguibile.

Il notebook dovrà essere autosufficiente: tutte le celle necessarie alla riproduzione dei risultati dovranno essere presenti e ordinate logicamente.

È richiesto che il codice sia leggibile, commentato e organizzato in funzioni o classi quando opportuno.

La valutazione terrà conto di:

- correttezza dell'implementazione;

- chiarezza della formulazione della loss;
- qualità dell'architettura GNN;
- completezza degli esperimenti;
- qualità dei plot e dell'analisi dei risultati;
- capacità di discutere criticamente limiti e possibili miglioramenti del metodo.

Suggerimenti operativi per gli studenti

1. Usare una loss direttamente sugli archi

Un modo semplice ed efficace per costruire la loss è sommare, su tutti gli archi, la sovrapposizione tra le distribuzioni di colore dei due nodi estremi:

$$\mathcal{L} = \sum_{(i,j) \in E} \sum_{k=1}^c p_{ik} p_{jk}.$$

Se due nodi adiacenti hanno alta probabilità sullo stesso colore, il contributo alla loss è grande. Se hanno probabilità concentrate su colori diversi, il contributo è piccolo.

2. Normalizzare la loss

Per confrontare grafi diversi o esperimenti con numeri diversi di archi, può essere utile usare:

$$\mathcal{L}_{\text{norm}} = \frac{1}{|E|} \sum_{(i,j) \in E} p_i \cdot p_j.$$

Allo stesso modo, è utile riportare l'errore normalizzato:

$$\epsilon = \frac{N_{\text{conflicts}}}{|E|}.$$

Questo rende più leggibili i risultati, soprattutto quando i due grafi hanno dimensioni diverse.

3. Separare loss continua e valutazione discreta

Durante il training la rete minimizza una loss continua. Tuttavia, il problema originale è discreto. È quindi importante distinguere:

- la loss differenziabile usata per il training;
- la colorazione discreta ottenuta con `argmax`;
- il numero effettivo di conflitti dopo discretizzazione.

Una loss bassa non garantisce automaticamente una colorazione valida: la misura decisiva è il numero di conflitti discreti.

4. Ripetere il training più volte

Il problema è non convesso e può avere molti minimi locali. È quindi consigliabile ripetere l'ottimizzazione più volte per lo stesso grafo e per lo stesso valore di c , cambiando il seed casuale.

Per ogni valore di c , gli studenti possono riportare:

- miglior numero di conflitti ottenuto;
- media e deviazione standard dei conflitti;
- frequenza con cui si ottiene una colorazione valida.

5. Cercare il numero cromatico con una scansione in c

Per stimare il numero cromatico, si può eseguire una scansione:

$$c = 2, 3, 4, \dots$$

e cercare il primo valore per cui il metodo trova una soluzione con zero conflitti.

Conviene non limitarsi a un solo training per valore di c , perché un fallimento potrebbe dipendere dall'ottimizzazione e non dall'impossibilità matematica di colorare il grafo.

6. Usare una baseline greedy

Come controllo, è utile confrontare il metodo GNN con una semplice euristica greedy implementata con NetworkX, per esempio `greedy_color`.

La baseline non deve necessariamente essere sofisticata, ma aiuta a capire se il metodo GNN sta producendo risultati ragionevoli.

7. Monitorare anche i conflitti durante il training

Oltre alla loss, può essere utile calcolare periodicamente la colorazione discreta con `argmax` e contare i conflitti.

Questo permette di produrre un plot del tipo:

$$N_{\text{conflicts}} \quad \text{vs} \quad \text{epoch}.$$

In alcuni casi la loss continua può ancora diminuire mentre il numero di conflitti discreti rimane costante.

8. Attenzione alle soluzioni “morbide”

Se la softmax rimane troppo distribuita sui colori, la rete può ridurre parzialmente la loss senza produrre una vera colorazione discreta buona.

Per mitigare questo problema si può:

- aggiungere una penalizzazione che favorisca distribuzioni più concentrate;
- monitorare l'entropia media delle distribuzioni di colore;
- aumentare il numero di epoche;
- provare learning rate diversi.

9. Possibile regolarizzazione entropica

Per incoraggiare assegnazioni più nette, si può aggiungere un termine che penalizzi l'entropia delle distribuzioni di colore:

$$H = -\frac{1}{|V|} \sum_i \sum_k p_{ik} \log p_{ik}.$$

Una possibile loss modificata è:

$$\mathcal{L}_{\text{tot}} = \mathcal{L}_{\text{Potts}} + \lambda H.$$

In questo caso, minimizzare H favorisce distribuzioni più vicine a vettori one-hot.

10. Usare un'architettura semplice ma ben controllata

Una possibile architettura minima è:

- embedding iniziale per ogni nodo;
- 2 o 3 layer di graph convolution/message passing;
- ReLU tra i layer;
- layer lineare finale con output di dimensione c ;
- softmax sui colori.

Non è necessario implementare una rete molto profonda. Anzi, reti troppo profonde possono rendere l'ottimizzazione più difficile.

11. Post-processing semplice

Se dopo il training rimangono pochi conflitti, si può provare una procedura di post-processing:

- identificare gli archi in conflitto;
- per uno dei due nodi coinvolti, provare a cambiare colore;
- accettare il cambiamento se riduce il numero di conflitti;
- ripetere finché possibile.

Un'altra possibilità, se si vuole stimare un upper bound sul numero cromatico, è aggiungere un colore extra e assegnarlo ad alcuni nodi coinvolti nei conflitti residui.

12. Visualizzare il grafo colorato

Per grafi piccoli o medi, è molto utile visualizzare il grafo finale con i nodi colorati secondo l'assegnazione trovata.

I conflitti residui possono essere evidenziati disegnando gli archi problematici con uno stile diverso.